

Relatório técnico - PIF- SI-2025.2

Jogo da Batalha Naval

Professor: João Victor Tinoco

Equipe:

Artur Nogueira
Eduardo Palhares
Matheus Arantes

1. Introdução

Este relatório descreve o processo de desenvolvimento, as decisões de design e a implementação técnica do jogo de tabuleiro "Batalha Naval", desenvolvido inteiramente na linguagem C.

Objetivo do Sistema: O sistema implementa o jogo de Batalha Naval para dois jogadores, gerenciando a configuração do tabuleiro, o posicionamento das frotas, o loop de turnos de tiro e a detecção da condição de vitória.

Foco Técnico: O projeto foi concebido com o objetivo primário de demonstrar o **domínio técnico** nos seguintes tópicos centrais da linguagem C, conforme exigido pela disciplina de PIF:

- Uso e implementação de **Structs** para modelagem de entidades (Board, Ship, Player).
- Utilização extensiva de **Ponteiros** para referências e acesso a memória.
- **Alocação Dinâmica de Memória** (usando **malloc**, **realloc** e **free**) para estruturas de dados flexíveis (tabuleiros e frotas).
- **Modularização** do código em arquivos de responsabilidade única para organização e manutenibilidade.

2. Arquitetura do Projeto

O código foi organizado seguindo um princípio de **separação de responsabilidades**, onde cada módulo (.c e .h) é responsável por uma parte distinta da lógica do jogo. O

código foi rigorosamente organizado com base no princípio de **modularização**, onde cada arquivo possui uma responsabilidade única e bem definida. Isso garantiu a clareza do projeto e facilitou a manutenção e a integração. O arquivo principal, **main.c**, serve como ponto de entrada e controle de alto nível, sendo responsável apenas por inicializar o ambiente do jogo (como a semente de aleatoriedade) e chamar as funções de ciclo de vida (**game_setup**, **game_loop** e **game_destroy**). A lógica central do jogo reside no módulo **game.c**. Ele gerencia o estado da partida, alternando os turnos, processando a lógica de tiro (**game_handle_shot**) e verificando a condição de vitória (**game_check_win**). O **game.c** atua como o orquestrador, chamando as funções dos outros módulos conforme necessário. O módulo **board.c** é a abstração do tabuleiro. Seu papel é gerenciar a estrutura de dados da matriz, incluindo a alocação e desalocação de memória dinâmica (**board_create**, **board_destroy**), e é o único responsável pela exibição visual do tabuleiro no terminal. O módulo **fleet.c** cuida de tudo relacionado aos navios. Ele é responsável por criar a frota, validar as regras complexas de posicionamento (**fleet_check_placement**) para evitar sobreposições e limites, e gerenciar o estado de dano e afundamento de cada navio. A interface com o usuário é isolada no módulo **io.c**. Sua função é garantir uma interação robusta, tratando menus, lendo *inputs* de forma segura (evitando "lixo" no *buffer*) e, crucialmente, convertendo coordenadas humanas (ex: "A5") para índices de matriz (linha, coluna), isolando a lógica de jogo da complexidade da entrada do usuário. Por fim, o módulo **rnd.c** encapsula todos os utilitários de números aleatórios, garantindo que o gerador de aleatoriedade seja inicializado corretamente apenas uma vez, mantendo a função **rand()** isolada e protegida. **Fluxo de Chamadas.** O fluxo de execução segue uma hierarquia clara, partindo do módulo principal e descendo para os módulos de lógica e utilitários.

3. Estruturas de Dados

A modelagem de dados usando **structs** e **malloc** foi o pilar do projeto. O relacionamento entre as entidades é feito primariamente através de ponteiros.

- **struct Board:**
 - Contém as dimensões (**rows**, **cols**).
 - Gerencia a matriz de células como um ponteiro único: **Cell *cells**.
 - **Funcionamento:** Isso permite alocação de memória contígua e acesso eficiente via aritmética de ponteiros, tratando o tabuleiro dinamicamente como um vetor 1D.
- **struct Ship:**

- Armazena o nome, o tamanho (**length**) e a vida atual (**hits**).
- **Funcionamento:** É a unidade de dano. A vida é decrementada a cada acerto. A condição de afundamento é quando **hits == length**.
- **struct Player:**
 - É uma estrutura agregadora. Contém:
 - **Board *real_board:** Onde os navios do jogador estão posicionados.
 - **Board *shot_board:** O tabuleiro que o jogador vê do inimigo (rastreado os tiros).
 - **Fleet *fleet:** O vetor dinâmico dos navios do jogador.
 - **Funcionamento:** Isola o estado de cada jogador, permitindo que a **struct Game** simplesmente gerencie dois **Players** (p1 e p2).
- **Relacionamento entre Frota e Tabuleiro:**
 - Ao posicionar um navio, a célula do **Board** é marcada com o **ship_id** (identificador único do navio) e o **CellState** como **CELL_SHIP**.
 - O **ship_id** é o elo: quando uma célula é atingida, o **game.c** usa o **ship_id** para localizar o **Ship** correspondente na **Fleet** do inimigo e incrementar seu contador de acertos (**hits**).
- **Uso e Controle de malloc:**
 - O **malloc** foi usado primariamente em **board_create** para alocar o bloco contíguo de **rows * cols** células.
 - **Controle de Memória:** Implementamos a alocação de memória **contígua** para garantir que todas as células possam ser acessadas por um único ponteiro (**Cell *cells**), facilitando a liberação com um único **free()** em **board_destroy**.
- **Garantia de Não Acesso Fora dos Limites:**
 - Todas as funções que acessam o tabuleiro (**board_get_cell**, **fleet_check_placement**) realizam uma **validação inicial** das coordenadas de entrada, garantindo que **row** e **col** estejam dentro dos limites de **0** a **rows-1** e **0** a **cols-1**, respectivamente.
 - Qualquer acesso via aritmética de ponteiros é precedido por essa checagem de limites.

4. Fluxo de Execução

O jogo segue uma sequência lógica bem definida, controlada principalmente pelo **main.c** e **game.c**.

1. Menu e Configurações (**io.c**):

- O jogador interage com o menu para escolher entre Novo Jogo, Carregar ou Sair.

- Se for um Novo Jogo, são lidas as configurações iniciais (nomes dos jogadores e, opcionalmente, o tamanho do tabuleiro).
- 2. **Posicionamento (fleet.c):**
 - O sistema itera sobre todos os navios definidos para o jogo.
 - Para cada navio, o jogador fornece coordenadas (A1, B5) e orientação (Horizontal/Vertical).
 - A função **fleet_check_placement** **valida** o posicionamento:
 - Checa limites do tabuleiro.
 - Checa sobreposição com outros navios (**CELL_WATER** obrigatório nas células de destino).
 - Se válido, as células do **Board** do jogador são marcadas com o **ship_id** e **CELL_SHIP**.
- 3. **Loop de Turnos (game.c):**
 - O jogo entra no *loop* principal, alternando entre Player 1 e Player 2.
 - **Ações do Turno:**
 - Exibição do tabuleiro de tiro do jogador atual (o que ele vê do inimigo).
 - Leitura de coordenada de tiro (**io.c**).
 - Validação da coordenada.
 - Processamento do Tiro (**game_handle_shot**): verifica o **real_board** do inimigo e atualiza o **shot_board** do jogador.
 - Atualização do estado do navio atingido na **Fleet** do inimigo.
- 4. **Deteccção de Vitória (game.c):**
 - Após cada tiro, a função **game_check_win** é chamada.
 - Ela itera sobre a **Fleet** do jogador que recebeu o tiro.
 - Se a vida (**hits**) de **todos** os navios for igual ao seu tamanho (**length**), o jogador atacante é declarado vencedor e o loop de turnos é encerrado.

5. Decisões de Design

Esta seção detalha as escolhas feitas para otimizar a estrutura e robustez do código.

- **Orientação Horizontal/Vertical (Representação):**
 - A validação de posicionamento foi baseada na **aritmética simples** ao invés de estruturas condicionais complexas.
 - Se a orientação for **Horizontal**, o algoritmo incrementa a coluna (**col++**).
 - Se a orientação for **Vertical**, o algoritmo incrementa a linha (**row++**).
 - Essa escolha simplificou o loop de verificação, utilizando uma única função de validação com base no índice a ser incrementado.
- **Validação da Coordenada (io.c):**

- Foi implementada uma função `io_converter_coord` que traduz a entrada do usuário ("B5") em índices `[row, col]` da matriz.
- **Escolha:** Optamos por esta conversão e validação na camada de I/O para garantir que a lógica de jogo (`game.c`) e os módulos de estrutura de dados (`board.c`, `fleet.c`) trabalhem **exclusivamente com índices numéricos**, isolando-os da complexidade da entrada humana (letras, números e *off-by-one* errors).
- **Uso de `rand()` em Módulo Separado (`rnd.c`):**
 - A função `srand(time(NULL))` foi encapsulada em `rnd_init()` dentro do `rnd.c`.
 - **Propósito:** Isso garante que a **semente do gerador de aleatoriedade seja inicializada apenas uma vez** (em `main.c`), prevenindo que múltiplas chamadas rápidas a `rand()` (ex: ao posicionar vários navios automaticamente) gerem sequências de números idênticos. A separação mantém o gerador de números aleatórios como um **serviço** limpo e centralizado.
- **Benefícios da Modularização:**
 - **Reuso:** Funções como `board_create` ou `io_read_coordinate` podem ser usadas por qualquer parte do código sem a necessidade de replicar a lógica.
 - **Teste Isolado:** Permitiu testar individualmente a alocação de memória (`board.c`) e a lógica de validação (`fleet.c`) antes de integrá-las ao fluxo de turnos.
 - **Clareza:** O `game.c` focou apenas na **lógica de alto nível** (o que acontece em um turno), delegando a exibição ao `io.c` e a gestão de navios ao `fleet.c`.

6. Gestão de Memória

A gestão de memória foi feita manualmente e com disciplina para evitar vazamentos, conforme exigido pelo foco em ponteiros.

- **Alocação (`malloc`/`realloc`) e `NULL` Check:**
 - **`malloc` para `Board`:** Em `board_create`, alocamos o bloco contíguo para as células.
 - **`realloc` para `Fleet`:** Demonstramos o uso de `realloc` na função `fleet_add_ship`. A frota inicia vazia (`NULL`) e é expandida em +1 posição cada vez que um navio é adicionado.
 - **Verificação:** Todas as chamadas de `malloc` e `realloc` são seguidas por um **teste de retorno `NULL`**. Se a alocação falhar, o programa encerra com uma mensagem de erro (`exit(EXIT_FAILURE)`).

- **Liberação (free) e Prevenção de Vazamento:**
 - Implementamos funções "espelho" de destruição: `board_destroy`, `fleet_destroy` e `game_destroy`.
 - A função `game_destroy` atua como um *destrutor* principal, liberando a memória em cascata: primeiro as frotas e tabuleiros de cada jogador, e depois a estrutura `Game` principal.
 - O `free()` é chamado no endereço base de cada bloco alocado por `malloc/realloc`, garantindo que toda a memória dinâmica seja devolvida ao sistema operacional.
- **Garantia de Não Acesso Fora do Vetor:**
 - O controle de limites, descrito na Seção 3, é a principal forma de prevenção.
 - Ao usar a aritmética de ponteiros para acessar células: `cells[ (row * cols) + col ]`, o sistema **confia** na validação de limites prévia, garantindo que o índice calculado esteja sempre dentro do bloco alocado contíguo.
- **Testes de Limites:**
 - Testes manuais de posicionamento foram focados em **extremos**: posicionar navios na linha/coluna 0 e na linha/coluna máxima.
 - O sistema confirmou que o navio não podia ser posicionado em `col > max_col - length`, demonstrando que o cálculo de offset estava correto.

7. Testes e Validação

A validação foi essencial para garantir a robustez do jogo, especialmente na manipulação de entradas do usuário e regras complexas de posicionamento.

- **Teste de Posicionamentos:**
 - **Posicionamento Válido:** Verificamos o sucesso ao posicionar navios em áreas vazias, testando ambas as orientações (horizontal/vertical) e todas as posições possíveis.
 - **Sobreposição (Inválido):** Foi testada a tentativa de posicionar um navio sobre células já ocupadas (`CELL_SHIP`). A função `fleet_check_placement` bloqueou corretamente a operação.
 - **Extravasamento (Inválido):** Tentamos posicionar navios que se estendiam além das bordas do tabuleiro (ex: colocar um navio de tamanho 4 começando na coluna 9 de um tabuleiro 10x10). O sistema validou o erro.
- **Teste de Coordenadas Válidas e Inválidas:**
 - Testamos a função `io_converter_coord` com:
 - **Válidas:** Entradas normais (ex: "A1", "J10").

- **Inválidas (Formato):** Entradas com erro de digitação (ex: "1A", "KK", "A11"). O tratamento de entrada (`io.c`) foi robusto o suficiente para rejeitar ou limpar essas entradas.
- **Verificação de Loop do Jogo:**
 - Foi garantido que o loop principal **não tranca** ao usar `fgets` para ler *strings* e limpar o buffer. Isso impediu que caracteres indesejados (`\n`) ficassem no buffer, causando pulos inesperados de *input*.
 - O loop só é interrompido pela condição de vitória.
- **Teste de Ataques Corretos e Incorretos:**
 - **Ataque Válido:** Atirar em uma célula de água ou em um navio. O estado do navio e do tabuleiro de tiro foi corretamente atualizado.
 - **Ataque Repetido (Inválido):** Tentamos atirar duas vezes na mesma coordenada. O sistema notificou o jogador e solicitou um novo tiro, garantindo que um tiro já registrado (`CELL_HIT` ou `CELL_MISS`) não fosse processado novamente.
 - **Condição de Afundamento:** Testamos a sequência exata de acertos necessária para afundar um navio, confirmando que a vida do navio é decretada corretamente e que a condição de vitória é detectada apenas quando a vida de **todos** os navios atinge zero.

8. Conclusão

Este projeto forneceu um valioso aprendizado prático sobre os conceitos fundamentais da linguagem C.

- **O que Aprendemos:**
 - O domínio da **Gestão de Memória Manual** com `malloc`, `realloc` e `free` foi o maior aprendizado, forçando-nos a pensar na alocação e desalocação como passos obrigatórios e interligados.
 - A **Modularização** com arquivos `.h` e `.c` demonstrou ser essencial para a escalabilidade e clareza de projetos maiores.
 - A importância de bibliotecas como `string.h` e `ctype.h` para construir **interfaces de usuário robustas**.
- **Quais Desafios Tivemos:**
 - O maior desafio técnico foi a implementação do `realloc` na `Fleet`, exigindo um cuidado extra com a manipulação do ponteiro base.
 - Garantir a **robustez da entrada/saída (I/O)**, especialmente o tratamento do buffer do teclado e a conversão de coordenadas, consumiu tempo significativo, mas resultou em um produto final mais estável.
- **O que Fariamos Melhor em Outra Versão:**

- Em uma próxima versão, integrariamos uma biblioteca de **testes unitários** (como CUnit) para automatizar a validação de funções críticas, como `fleet_check_placement` e `io_converter_coord`, em vez de depender apenas de testes manuais.
- Melhorar a experiência de `Player` com um sistema de salvar/carregar o jogo em disco (utilizando `fwrite` e `fread`).